

Viva: Project 17, SpaceX Launch Tracker

API: SpaceX API (<https://github.com/r-spacex/SpaceX-API>) - Key: No key needed

Brief: Past and upcoming SpaceX launches with mission details.

30 questions, roughly easy to hard. Sections A and B are common ground for every project; C and D are specific to this one. Ask 8 to 12 in a typical 10 minute viva.

A. Kotlin and Android basics

1. What is the difference between `val` and `var`, and which did you use for your Retrofit instance?
2. What is a data class, and what does Kotlin generate for you?
3. How did you model a launch, and which of its many fields did you keep?
4. What does `?` mean on a property, and which launch fields are genuinely optional?
5. How do you pass the selected launch to the detail screen?
6. How did you show past and upcoming launches, two lists or tabs?
7. Where do you declare the `INTERNET` permission, and what happens if you forget it?

B. Retrofit, coroutines and networking

8. What problem does Retrofit solve for you?
9. Explain the role of the converter factory in your Retrofit builder.
10. Which endpoints did you call, and did any of them need a POST body?
11. Why is your API call function marked `suspend`?
12. Which dispatcher runs the network call, and which updates the UI?
13. What happens if you call the API on the main thread?
14. What is a coroutine scope, and which one did you launch from?
15. How does your code behave when the list loads but a detail request fails?

C. The SpaceX API and this app

16. Which version of the API did you use, v4 or v5, and what is the base URL?
17. The v4 query endpoint uses POST with a JSON body for filtering and paging. Did you use it, and how does that differ from a normal GET?
18. `date_utc` and `date_local` are both returned. Which did you display and why?
19. `date_precision` can be hour, day, month or year. What does that mean for showing an upcoming launch time?
20. `success` is null for upcoming launches, true or false for past ones. How does your UI handle all three states?
21. `Rocket` and `launchpad` come back as ID strings, not objects. How did you turn an ID into a readable name?
22. What does the `populate` option do in the query endpoint, and how would it save you requests?
23. Patch images live under `links.patch.small` and `large`. How did you model that nesting?
24. Many older launches have no patch image. What does your app show instead?

D. Edge cases, design and extensions

25. What does your app show while launches are loading?
26. A user has no connection and opens the app. Describe exactly what they see.
27. How did you sort launches, and does the API sort for you or do you sort locally?
28. How would you add a countdown timer to the next launch?
29. How would you cache launches so the app opens instantly?
30. How would you add the webcast link so a user can watch the launch?

Humble Coders project viva preparation. These are the kinds of questions you should be able to answer about the app you built. Knowing the list is not the same as knowing your own code.